

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

Sistema Web MVC para Gestion de Inventario y Ventas de Panaderia

Nombre del Proyecto	Panderia
Estructura Tridimensional	Sistema Web (Que) Arquitectura MVC (Como) Gestion de Inventario y Ventas de Panaderia (Donde)
Version SAD	1.0 - Mayo 2026
Arquitecto Responsable	Equipo de Desarrollo
Equipo	Paola Jaqueline López Mata Gerardo Manzano Villafañá Jennifer Ailin Medina Hernández

1. Introduccion y Objetivos Tecnicos

PanTrack System es una plataforma web que implementa el patron Modelo-Vista-Controlador (MVC) para gestionar el ciclo completo de ventas e inventario de una panaderia: alta de productos y materias primas, registro de ventas con descuento automatico de stock, control de turnos y empleados, retiros de caja, sistema de alertas de reorden y generacion de reportes.

Objetivos tecnicos principales:

- Proveer una API RESTful modular como backend desacoplado del frontend.
- Garantizar consistencia de datos mediante transacciones ACID en operaciones de venta y retiro de caja.
- Mantener una arquitectura mantenible con separacion clara de capas (Model / View / Controller).
- Gestionar productos segmentados en tres categorias operativas: panaderia, refrigerados y tienda.
- Controlar turnos de empleados y registrar retiros de caja por turno.
- Habilitar despliegue con cero friccion mediante contenedores Docker.

2. Requerimientos No Funcionales (Atributos de Calidad)

Atributo	Requerimiento	Metrica / Mecanismo
Escalabilidad	Soportar hasta 50 usuarios concurrentes en fase inicial	Pruebas de carga con Artillery; escalar horizontalmente con Docker Compose
Disponibilidad	Uptime del 99.5% en horario operativo (6am-10pm)	Health-check endpoint; reinicio automatico de contenedor en fallo
Seguridad	Autenticacion con JWT; datos sensibles cifrados	Tokens con expiracion 8h; HTTPS obligatorio; bcrypt para contraseñas
Rendimiento	Respuesta de API < 400ms para el 95% de peticiones	Indices en tablas productos, ventas e inventario_diario; cache Redis para consultas frecuentes
Mantenibilidad	Codigo modular, cobertura de pruebas >= 80%	ESLint + Prettier; Jest para unitarias; GitHub Actions para CI
Portabilidad	Funcionar en cualquier entorno con Docker instalado	docker-compose.yml versionado en repositorio

3. Vista Logica - Arquitectura de Alto Nivel

3.1 Patron Arquitectonico: MVC Modular

Se adopta el patron Modelo-Vista-Controlador (MVC) con separacion explicita de responsabilidades en tres capas:

Capa	Responsabilidad	Tecnologia
MODEL	Definicion de entidades (12 tablas), reglas de negocio y acceso a BD. ORM sobre PostgreSQL.	Sequelize ORM + PostgreSQL 16
VIEW	Interfaz de usuario. Consume la API REST del backend. Sin logica de negocio.	React 18 + Vite
CONTROLLER	Maneja peticiones HTTP, orquesta modelos y retorna respuestas JSON.	Node.js 20 + Express.js + TypeScript
SERVICIOS	Logica de negocio compleja: calculo de alertas, generacion de reportes, gestion de turnos.	Service Layer dentro del backend
PERSISTENCIA	Almacenamiento relacional principal con 12 tablas normalizadas.	PostgreSQL 16
CACHE	Sesiones y consultas de alta frecuencia (precios, stock).	Redis 7

3.2 Flujo de Datos Principal (Registro de Venta)

El siguiente flujo describe el proceso central del sistema:

1. El Cajero selecciona productos y cantidad en la interfaz React (VIEW).
2. El frontend realiza una peticion POST /api/ventas al backend Express (CONTROLLER).
3. El controlador valida el JWT del usuario y el turno activo del empleado.
4. El VentaService verifica stock disponible en productos, productos_refri o productos_tienda segun categoria.
5. Dentro de una transaccion ACID: crea el registro en ventas, inserta los registros en detalle_venta y descuenta el stock.
6. Se registra el movimiento en inventario_diario para trazabilidad.
7. Se retorna respuesta JSON con folio de venta; el dashboard del cajero se actualiza.

4. Stack Tecnologico Autorizado

Capa / Componente	Tecnologia	Version	Razon de Eleccion
Frontend	React + Vite	React 18.x	SPA rapida; ecosistema amplio; sin SSR necesario en fase inicial
Backend (Controller)	Node.js + Express + TypeScript	Node 20 LTS	Agilidad, tipado fuerte, amplia comunidad y librerias REST
ORM (Model)	Sequelize	6.x	Migracion sencilla, compatible con PostgreSQL, soporte de transacciones
Base de Datos Principal	PostgreSQL	16.x	Relacional; transacciones ACID; ideal para inventario con consistencia critica
Cache / Sesiones	Redis	7.x	Velocidad en consultas frecuentes; gestion de tokens de sesion
Autenticacion	JWT + bcrypt	jsonwebtoken 9.x	Estandar sin estado; facil integracion en API REST
Contenedores	Docker + Docker Compose	Ultima LTS	Homogeneidad entre entornos local y produccion
Control de Versiones	Git + GitHub	Git 2.x	Versionado, colaboracion y despliegue en GitHub Pages
CI/CD	GitHub Actions	N/A	Automatizacion de pruebas en cada push al repositorio
Pruebas Unitarias	Jest + Supertest	Jest 29.x	Cobertura de rutas API y logica de negocio
Linting / Formato	ESLint + Prettier	ESLint 8.x	Estandares de codigo homogeneos en el equipo
Generacion de Reportes	jsPDF / ExcelJS	Ultima estable	Exportacion de reportes a PDF y CSV/XLSX desde el backend

5. Modelo de Datos - Entidades Principales (12 Tablas)

Las siguientes entidades conforman el esquema relacional de PanTrack System. El modelo esta diseñado para soportar tres tipos de productos con ciclos de vida distintos, control de turnos y trazabilidad completa de inventario.

5.1 Catalogo de Productos

Tabla	Atributos Clave	Relaciones
categorias_pan	id, nombre, descripcion, activo	Agrupar a productos (panaderia)
productos	id, nombre, precio_unitario, stock_actual, categoria_id, activo	Pertenece a categorias_pan; aparece en detalle_venta; tiene historico_precios
productos_refri	id, nombre, precio_unitario, stock_actual, temperatura_min, activo	Aparece en detalle_venta; tiene historico_precios
productos_tienda	id, nombre, precio_unitario, stock_actual, proveedor, activo	Aparece en detalle_venta; tiene historico_precios

5.2 Ventas e Inventario

Tabla	Atributos Clave	Relaciones
ventas	id, fecha, total, folio, usuario_id, turno_id, estado	Tiene muchos detalle_venta; pertenece a usuarios y turnos
detalle_venta	id, venta_id, producto_id, tipo_producto (pan/refri/tienda), cantidad, precio_unitario	Pertenece a ventas; referencia productos, productos_refri o productos_tienda segun tipo
inventario_diario	id, fecha, producto_id, tipo_producto, stock_apertura, entradas, salidas, stock_cierre	Registro diario de movimientos de stock por producto
historico_precios	id, producto_id, tipo_producto, precio_anterior, precio_nuevo, fecha_cambio, usuario_id	Audita cambios de precio para cualquier tipo de producto

5.3 Operacion y Personal

Tabla	Atributos Clave	Relaciones
usuarios	id, nombre, email, password_hash, rol (admin/cajero), activo	Tiene muchas ventas; registra cambios de precio; abre/cierra turnos
empleados	id, nombre, puesto, telefono, usuario_id, activo	Vinculado a usuarios; asociado a turnos
turnos	id, empleado_id, fecha_apertura, fecha_cierre, fondo_inicial, total_ventas, estado (abierto/cerrado)	Agrupar ventas del periodo; tiene retiros_caja; pertenece a empleados
retiros_caja	id, turno_id, monto, motivo, usuario_id, fecha	Pertenece a turnos; registrado por usuarios

5.4 Resumen de Relaciones Clave

Relacion	Cardinalidad	Descripcion
categorias_pan -> productos	1 : N	Una categoria agrupa N productos de panaderia
productos / productos_refri / productos_tienda -> detalle_venta	1 : N (polim.)	Un producto de cualquier tipo aparece en N detalles; el campo tipo_producto discrimina la tabla origen
ventas -> detalle_venta	1 : N	Una venta contiene N renglones de detalle
turnos -> ventas	1 : N	Un turno agrupa todas las ventas del periodo
turnos -> retiros_caja	1 : N	Un turno puede tener N retiros de caja
empleados -> turnos	1 : N	Un empleado puede tener N turnos en el tiempo
usuarios -> empleados	1 : 1	Cada empleado tiene exactamente un usuario del sistema
productos -> historico_precios	1 : N (polim.)	Cualquier tipo de producto puede tener N registros de cambio de precio
productos -> inventario_diario	1 : N (polim.)	Cada producto tiene un registro diario de movimientos de stock

6. Estrategia de Despliegue - GitHub Pages

El sistema se desplegara mediante un repositorio Git unico compartido por el equipo:

- Un solo repositorio por equipo con estructura /frontend y /backend.
- Los documentos PID y SAD se publicaran como sitio estatico en GitHub Pages (rama gh-pages).
- URL publica: [https://\[org-github\].github.io/pantrack-system](https://[org-github].github.io/pantrack-system)
- GitHub Actions ejecuta linting y pruebas en cada push; el despliegue se activa solo si el pipeline pasa.
- Variables de entorno sensibles gestionadas mediante GitHub Secrets (nunca en el repositorio).